

课题05-Triton-attention-kernel

Contents

1 课题05 开题简报 · 手写 Triton attention kernel	1
1.1 一、研究背景	1
1.2 二、研究问题（一句话）	2
1.3 三、攻关线索	2
1.4 四、里程碑	2
1.5 五、交付物	2
1.6 六、讲课锚点	3
1.7 七、代码级提问示例	3
1.8 八、风险与提醒	3

1 课题05 开题简报 · 手写 Triton attention kernel

模块：M5 | 周次：W7-W8 (10-26→11-08) | 算力：T1
Kaggle 1×T4, 计划 10 GPU·小时 唐杰作业对应：② 手写
Triton attention kernel, 测多卡训练与推理增益 | 判分来源
：CS336 A2 (正确性对齐) + 自建性能对照 手写：R5 (kernel
本体：分块 + online softmax)

1.1 一、研究背景

朴素注意力要把 $L \times L$ 的注意力矩阵整个写进显存： $L = 8192$ 时单头就是 64M 个元素。FlashAttention (2022) 的核心洞察：根本不需要把矩阵写出来——分块计算、在线维护 softmax 的最大值与归一化因子，就能得到数学上完全等价的结果。它把注意力的显存从 $O(L^2)$ 降到 $O(L)$ 。

Triton 让这件事对非 CUDA 专家也可做：用 Python 风格的 DSL 写”块级”程序。本课题的目标不是写出最快的 kernel，而是亲手经历”数值等价 + 显存下降 + 速度提升”这三件事的因果链。

1.2 二、研究问题（一句话）

一个分块 + **online softmax** 的手写 **kernel**，能否在数值上等价于朴素实现，并把显存从 $O(L^2)$ 降到 $O(L)$ 、速度提升若干倍？提升来自哪里（算术减少？访存减少？）

1.3 三、攻关线索

1. 先量化瓶颈：朴素实现在 $L=2k/8k$ 时的显存、耗时（用 profiler 看是 compute-bound 还是 memory-bound）
2. **online softmax** 的递推：维护 running max m 与 running sum ℓ ，新块的贡献要按 $e^{m_{old}-m_{new}}$ 缩放 —— 这正是 M0 §四 logsumexp 的应用
3. 分块维度：按 query 分块、沿 key 方向扫描 → 每个块只需常驻 SRAM
4. 数值容差：与 PyTorch SDPA 对照，相对误差应在 $1e-2 \sim 1e-3$ (fp16)；先写容差，再判定
5. 多卡/推理增益：训练侧测吞吐 (tokens/s)，推理侧测 decode 显存与延迟 —— 这是唐杰作业②要求的“增益”

1.4 四、里程碑

- M1 基线：朴素注意力的显存与耗时 ($L=2k/4k/8k$ 三点)
- M2 正确性：手写 kernel 与 SDPA 的数值对照 (max abs error 记录在案)
- M3 显存：证明峰值显存随 L 近似线性 (对比朴素实现的 $O(L^2)$)
- M4 速度：给出 speedup 曲线 (含 L 与 block size 两个维度)
- M5 **profiling**：用 profiler 说明“提升来自哪里” (访存 vs 算术，给出数字)
- M6 增益测量：训练吞吐 (tokens/s) 与推理 (融合进 attention 后) 的实测对比
- M7 一页报告 + 知识卡 (online softmax 递推必须能徒手推)

1.5 五、交付物

交付物	位置	验收
kernel	手写/ (你写)	M2 数值对照通过
三张图	证据/	speedup / 显存 / profile (坐标轴与硬件口径必须标注)
数值误差记录	证据/	含容差设定与依据

交付物	位置	验收
报告	报告.md	必须回答”提升来自哪里”

1.6 六、讲课锚点

- 讲义：讲义/M5-推理.md (W5 展开) 中”GPU 内存层级与 Triton 编程模型”一节
- 对标：CS336 lecture 05-06 (GPUs / Kernels & Triton) + A2 hand-out (**flash-attn** 需两步安装： `uv sync --no-install-package flash-attn` 再 `uv sync`) · Triton 官方 FlashAttention 教程
- 搭桥：M0 §四 的 `logsumexp` —— 你会亲手把它写进 kernel

1.7 七、代码级提问示例

1. online softmax 里，为什么新块要乘 $e^{m_{\text{old}} - m_{\text{new}}}$ ？不乘会怎样？
2. block size 从 64 增到 256：显存、速度、数值误差各往哪走？
3. 为什么 L 很大时，朴素实现容易 OOM 而 kernel 不会？（用 $O(L^2)$ vs $O(L)$ 说清）
4. 【下注】你预测 speedup 在 L=2k 时是多少倍？先写，再测。

1.8 八、风险与提醒

- 🚫 **P100** 不可用于本课题：Triton 要求计算能力 ≥ 7.0 ，P100 是 6.0 → 开跑前必须验明是 **T4**。
- △ 正确性优先于速度：数值对照不过，速度数字一律不算数。
- △ 硬件口径陷阱：任何 speedup 数字必须写明”对比对象 (PyTorch SDPA / 朴素实现) “与”硬件 (T4) “， 否则曲线毫无意义（这一条写进报告硬要求）。
- △ 若 T4 上 kernel 因共享内存不够而失败，先降 **block size**，不要放弃（这本身就是教学素材）。